

Solution Requirements

Date	01 July 2026
Team ID	
Project Name	HDI Predictor using Machine Learning and Flask
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	<i>Not required in the current iteration. The application is public-facing and accessible without authentication. If expanded, it should support standard session-based user authentication.</i>	Low
2	Authorization levels	<i>Two operational roles: 1. General User: Can run single predictions, upload batch CSV files, and view/download charts. 2. Administrator: Can access health metrics, check server statuses, and upload updated model</i>	Medium

		<i>pickle files (HDI.pkl).</i>	
3	External interfaces	<i>The system must query content CDNs (unpkg, cdnjs) to fetch and render Tailwind CSS stylesheets and Chart.js visualization libraries, and fetch custom fonts from the Google Fonts API.</i>	High
4	Transactions processing	<i>The backend must ingest drag-and-drop CSV payloads, validate matching schema headers, compute normalized indicator values, execute model prediction runs, and return compiled JSON payloads mapping inputs to results.</i>	High
5	Reporting	<i>The client interface must render dynamic radar charts representing the three sub-indices (life expectancy index, education index, and GNI index) and bar charts representing indicator deviations using Chart.js</i>	High
6	Business rules, etc.	<i>1. Input parameter ranges must satisfy boundary limits: Life expectancy (0–120), Expected schooling (0–25), Mean schooling (0–25), GNI per capita (>0).
2. Target HDI classifications: Very High (\$\ge 0.8\$), High (\$0.7 -</i>	High

		0.799\$), Medium (\$0.55 - 0.699\$), Low (\$< 0.55\$).	
7	Compliance to laws or regulations	<i>Complies with GDPR/CCPA. The prediction endpoints must process and release all calculation variables strictly in memory; no PII (Personally Identifiable Information) or user IP address data is logged or retained</i>	Low
8	Other	<i>The system must execute validation runs on inputs (bounds check, null checking, type checks) and return human-readable validation errors instead of throwing server exceptions.</i>	High

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	<i>The application should deliver instant response times for predictions and render charts with minimal delay.</i>	<i>Page load time \$< 2\$ seconds on a 3G network; single prediction API response time \$< 100\$ milliseconds.</i>

2	Scalability	<i>The backend server must process concurrent prediction requests during load spikes.</i>	<i>Support up to \$1,000\$ concurrent prediction requests when deployed with a production WSGI container (e.g., Gunicorn)</i>
3	Security & Data Privacy	<i>Secure communication and input filtering to block vulnerability exploits.</i>	<i>Production deployment over HTTPS; strict validation to block cross-site scripting (XSS) and injection vectors via input forms.</i>
4	Reliability & Availability	<i>Continuous availability of prediction features.</i>	<i>99.5% uptime per month; fallback functionality to load default indices if the country lookup database file is missing.</i>
5	Usability & Accessibility	<i>Responsive layout across mobile, tablet, and desktop screens with clear visual contrast.</i>	<i>Full page usability on screens from 320px to 1920px wide; WCAG 2.1 AA compliant contrast ratios.</i>
6	Other	<i>Modular structure allowing easy model updates or front-end redesigns.</i>	<i>Clear separation of concerns between model prediction routines ('predictor.py') and UI routing Controllers ('routes/').</i>